How to Configure the Learning Rate When Training Deep Learning Neural Networks

by **Jason Brownlee** on January 23, 2019 in **Deep Learning Performance**

Last Updated on August 6, 2019

The weights of a neural network cannot be calculated using an analytical method. Instead, the weights must be discovered via an empirical optimization procedure called stochastic gradient descent.

The optimization problem addressed by stochastic gradient descent for neural networks is challenging and the space of solutions (sets of weights) may be comprised of many good solutions (called global optima) as well as easy to find, but low in skill solutions (called local optima).

The amount of change to the model during each step of this search process, or the step size, is called the "*learning rate*" and provides perhaps the most important hyperparameter to tune for your neural network in order to achieve good performance on your problem.

In this tutorial, you will discover the learning rate hyperparameter used when training deep learning neural networks.

After completing this tutorial, you will know:

- Learning rate controls how quickly or slowly a neural network model learns a problem.
- How to configure the learning rate with sensible defaults, diagnose behavior, and develop a sensitivity analysis.
- How to further improve performance with learning rate schedules, momentum, and adaptive learning rates.

Kick-start your project with my new book Better Deep Learning, including *step-by-step tutorials* and the *Python source code* files for all examples.

Let's get started.

How to Configure the Learning Rate Hyperparameter When Training Deep Learning Neural Networks
Photo by Bernd Thaller, some rights reserved.

Tutorial Overview

This tutorial is divided into six parts; they are:

1. What Is the Learning Rate?
2. Effect of Learning Rate
3. How to Configure Learning Rate
4. Add Momentum to the Learning Process
5. Use a Learning Rate Schedule
6. Adaptive Learning Rates

What Is the Learning Rate?

Deep learning neural networks are trained using the stochastic gradient descent algorithm.

Stochastic gradient descent is an optimization algorithm that e… the model using examples from the training dataset, then updates … tion of errors algorithm, referred to as simply backpropagation.

The amount that the weights are updated during training is referred to as the step size or the "*learning rate*."

Specifically, the learning rate is a configurable hyperparamete… small positive value, often in the range between 0.0 and 1.0.

> *… learning rate, a positive scalar determining the size*

— Page 86, Deep Learning, 2016.

The learning rate is often represented using the notation of the…

During training, the backpropagation of error estimates the am… e network are responsible. Instead of updating the weight with t…

This means that a learning rate of 0.1, a traditionally common default value, would mean that weights in the network are updated 0.1 * (estimated weight error) or 10% of the estimated weight error each time the weights are updated.

Effect of Learning Rate

A neural network learns or approximates a function to best map inputs to outputs from examples in the training dataset.

The learning rate hyperparameter controls the rate or speed at which the model learns. Specifically, it controls the amount of apportioned error that the weights of the model are updated with each time they are updated, such as at the end of each batch of training examples.

Given a perfectly configured learning rate, the model will learn to best approximate the function given available resources (the number of layers and the number of nodes per layer) in a given number of training epochs (passes through the training data).

Generally, a large learning rate allows the model to learn faster, at the cost of arriving on a sub-optimal final set of weights. A smaller learning rate may allow the model to learn a more optimal or even globally optimal set of weights but may take significantly longer to train.

At extremes, a learning rate that is too large will result in weight updates that will be too large and the performance of the model (such as its loss on the training dataset) will oscillate over training epochs. Oscillating performance is said to be caused by weights that diverge (are divergent). A le or may get stuck on a suboptimal solution.

> *When the learning rate is too large, gradient descent can inadvertently increase rather than decrease the training error. […] When the learning rate is too small, training is not only slower, but may become permanently stuck with a high training error.*

— Page 429, Deep Learning, 2016.

In the worst case, weight updates that are too large may caus overflow).

> *When using high learning rates, it is possible to encou weights induce large gradients which then induce a lar consistently increase the size of the weights, then [the numerical overflow occurs.*

— Page 238, Deep Learning, 2016.

Therefore, we should not use a learning rate that is too large or too small. Nevertheless, we must configure the model in such a way that on average a "*good enough*" set of weights is found to approximate the mapping problem as represented by the training dataset.

How to Configure Learning Rate

It is important to find a good value for the learning rate for your model on your training dataset.

The learning rate may, in fact, be the most important hyperparameter to configure for your model.

> *The initial learning rate […] This is often the single most important hyperparameter and one should always make sure that it has been tuned […] If there is only time to optimize one hyper-parameter and one uses stochastic gradient descent, then this is the hyper-parameter that is worth tuning*

— Practical recommendations for gradient-based training of deep architectures, 2012.

In fact, if there are resources to tune hyperparameters, much of this time should be dedicated to tuning the learning rate.

> *The learning rate is perhaps the most important hyperparameter. If you have time to tune only one hyperparameter, tune the learning rate.*

— Page 429, Deep Learning, 2016.

Unfortunately, we cannot analytically calculate the optimal learning rate for a given model on a given dataset. Instead, a good (or good enough) learning rate must be discovered via trial and error.

> *… in general, it is not possible to calculate the best learning rate a priori.*

— Page 72, Neural Smithing: Supervised Learning in Feedforward Artificial Neural Networks, 1999.

The range of values to consider for the learning rate is less tha

> *Typical values for a neural network with standardized inputs (or inputs mapped to the (0,1) interval) are less than 1 and greater than 10^−6*

— Practical recommendations for gradient-based training of d

The learning rate will interact with many other aspects of the c
nonlinear. Nevertheless, in general, smaller learning rates will
learning rates will require fewer training epochs. Further, smal g
rates given the noisy estimate of the error gradient.

A traditional default value for the learning rate is 0.1 or 0.01, a ur
problem.

> *A default value of 0.01 typically works for standard multi-layer neural networks but it would be foolish to rely exclusively on this default value*

— Practical recommendations for gradient-based training of deep architectures, 2012.

Diagnostic plots can be used to investigate how the learning rate impacts the rate of learning and learning dynamics of the model. One example is to create a line plot of loss over training epochs during training. The line plot can show many properties, such as:

- The rate of learning over training epochs, such as fast or slow.
- Whether model has learned too quickly (sharp rise and plateau) or is learning too slowly (little or no change).
- Whether the learning rate might be too large via oscillations in loss.

Configuring the learning rate is challenging and time-consuming.

> *The choice of the value for [the learning rate] can be fairly critical, since if it is too small the reduction in error will be very slow, while, if it is too large, divergent oscillations can result.*

— Page 95, Neural Networks for Pattern Recognition, 1995.

An alternative approach is to perform a sensitivity analysis of the learning rate for the chosen model, also called a grid search. This can help to both highlight an order of magnitude where good learning rates may reside, as well as describe the relationship between learning rate and performance.

It is common to grid search learning rates on a log scale from 0.1 to 10^-5 or 10^-6.

> *Typically, a grid search involves picking values approximately on a logarithmic scale, e.g., a learning rate taken within the set {.1, .01, 10−3, 10−4 , 10−5}*

— Page 434, Deep Learning, 2016.

When plotted, the results of such a sensitivity analysis often show a "U" shape, where loss decreases (performance improves) as the learning rate is decreased with a fixed number of training epochs to a point where loss sharply increases again because the model fails to converge.

If you need help experimenting with the learning rate for your

- Understand the Impact of Learning Rate on Model Performance With Deep Learning Neural Networks

Add Momentum to the Learning Process

Training a neural network can be made easier with the additio

Specifically, an exponentially weighted average of the prior up
weights are updated. This change to stochastic gradient desc
update procedure, causing many past updates in one directio

> *The momentum algorithm accumulates an exponentia*
> *continues to move in their direction.*

— Page 296, Deep Learning, 2016.

Momentum can accelerate learning on those problems where the high-dimensional "*weight space*" that is being navigated by the optimization process has structures that mislead the gradient descent algorithm, such as flat regions or steep curvature.

> *The method of momentum is designed to accelerate learning, especially in the face of high curvature, small but consistent gradients, or noisy gradients.*

— Page 296, Deep Learning, 2016.

The amount of inertia of past updates is controlled via the addition of a new hyperparameter, often referred to as the "*momentum*" or "*velocity*" and uses the notation of the Greek lowercase letter alpha (*a*).

> *… the momentum algorithm introduces a variable v that plays the role of velocity — it is the direction and speed at which the parameters move through parameter space. The velocity is set to an exponentially decaying average of the negative gradient.*

— Page 296, Deep Learning, 2016.

It has the effect of smoothing the optimization process, slowing updates to continue in the previous direction instead of getting stuck or oscillating.

> *One very simple technique for dealing with the problem of widely differing eigenvalues is to add a momentum term to the gradient descent formula. This effectively adds inertia to the motion through weight space and smoothes out the oscillations*

— Page 267, Neural Networks for Pattern Recognition, 1995.

Momentum is set to a value greater than 0.0 and less than one, where common values such as 0.9 and 0.99 are used in practice.

> *Common values of [momentum] used in practice include .5, .9, and .99.*

— Page 298, Deep Learning, 2016.

Momentum does not make it easier to configure the learning rate, as the step size is independent of the momentum. Instead, momentum can improve the speed of the optimization process in concert with the step size, improving the likelihood that a better set of weights is discovered in fewer training epochs.

Use a Learning Rate Schedule

An alternative to using a fixed learning rate is to instead vary t

The way in which the learning rate changes over time (training ule or learning rate decay.

Perhaps the simplest learning rate schedule is to decrease the a small value. This allows large weight changes in the beginning e-tuning towards the end of the learning process.

> *In practice, it is necessary to gradually decrease the learning rate over time, so we now denote the learning rate at iteration […] This is because the SGD gradient estimator introduces a source of noise (the random sampling of m training examples) that does not vanish even when we arrive at a minimum.*

— Page 294, Deep Learning, 2016.

In fact, using a learning rate schedule may be a best practice when training neural networks. Instead of choosing a fixed learning rate hyperparameter, the configuration challenge involves choosing the initial learning rate and a learning rate schedule. It is possible that the choice of the initial learning rate is less sensitive than choosing a fixed learning rate, given the better performance that a learning rate schedule may permit.

The learning rate can be decayed to a small value close to zero. Alternately, the learning rate can be decayed over a fixed number of training epochs, then kept constant at a small value for the remaining training epochs to facilitate more time fine-tuning.

> *In practice, it is common to decay the learning rate linearly until iteration [tau]. After iteration [tau], it is common to leave [the learning rate] constant.*

— Page 295, Deep Learning, 2016.

Adaptive Learning Rates

The performance of the model on the training dataset can be monitored by the learning algorithm and the learning rate can be adjusted in response.

This is called an adaptive learning rate.

Perhaps the simplest implementation is to make the learning rate smaller once the performance of the model plateaus, such as by decreasing the learning rate by a factor of two or an order of magnitude.

> *A reasonable choice of optimization algorithm is SGD with momentum with a decaying learning rate (popular decay schemes that perform better or worse on different problems include decaying linearly until reaching a fixed minimum learning rate, decaying exponentially, or decreasing the learning rate by a factor of 2-10 each time validation error plateaus).*

— Page 425, Deep Learning, 2016.

Alternately, the learning rate can be increased again if performance does not improve for a fixed number of training epochs.

An adaptive learning rate method will generally outperform a r

> *The difficulty of choosing a good learning rate a priori i methods are so useful and popular. A good adaptive a simple back-propagation with a poorly chosen fixed lea*

— Page 72, Neural Smithing: Supervised Learning in Feedfor

Although no single method works best on all problems, there are three adaptive learning rate methods that have proven to be robust over many types of neural network architectures and problem types.

They are AdaGrad, RMSProp, and Adam, and all maintain and adapt learning rates for each of the weights in the model.

Perhaps the most popular is Adam, as it builds upon RMSProp and adds momentum.

> *At this point, a natural question is: which algorithm should one choose? Unfortunately, there is currently no consensus on this point. Currently, the most popular optimization algorithms actively in use include SGD, SGD with momentum, RMSProp, RMSProp with momentum, AdaDelta and Adam.*

— Page 309, Deep Learning, 2016.

A robust strategy may be to first evaluate the performance of a model with a modern version of stochastic gradient descent with adaptive learning rates, such as Adam, and use the result as a baseline. Then, if time permits, explore whether improvements can be achieved with a carefully selected learning rate or simpler learning rate schedule.

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Post

- Understand the Impact of Learning Rate on Model Performance With Deep Learning Neural Networks

Papers

- Practical recommendations for gradient-based training of deep architectures, 2012.

Books

- Chapter 8: Optimization for Training Deep Models, Deep Learning, 2016.
- Chapter 6: Learning Rate and Momentum, Neural Smithing: Supervised Learning in Feedforward Artificial Neural Networks, 1999.
- Section 5.7: Gradient descent, Neural Networks for Pattern Recognition, 1995.

Articles

- Stochastic gradient descent, Wikipedia.
- What learning rate should be used for backprop?, Neural Network FAQ.

Summary

In this tutorial, you discovered the learning rate hyperparamet ks.

Specifically, you learned:

- Learning rate controls how quickly or slowly a neural netw
- How to configure the learning rate with sensible defaults, analysis.
- How to further improve performance with learning rate sc

Do you have any questions?
Ask your questions in the comments below and I will do my best to answer.

About Jason Brownlee

Jason Brownlee, PhD is a machine learning specialist who teaches developers how to get results with modern machine learning methods via hands-on tutorials.
View all posts by Jason Brownlee →

‹ How to Control the Stability of Training Neural Networks With the Batch Size

Understand the Impact of Learning Rate on Neural Network Performance ›

47 Responses to *How to Configure the Learning Rate When Training Deep Learning Neural Networks*

Dennis Cartin January 23, 2019 at 6:10 pm #

Hi Jason

As always great article and worth reading.

Adam has this Adam(lr=0.001, beta_1=0.9, beta_2=0.999, epsi
in Adam the implementation of adapted learning rates.

Are we going to create our own class and callback to implemer

Thanks
Dennis

Jason Brownlee January 24, 2019 at 6:43 am # REPLY

The Adam algorithm implements the adaptive learning rate itself, one for each parameter in the model.

More details here:
https://machinelearningmastery.com/adam-optimization-algorithm-for-deep-learning/

Dennis Cartin January 31, 2019 at 8:35 pm # REPLY

Thanks Jason for the feedback.

BTW, I have one question not related on this post. I am wondering on my recent model in keras. After cross validation of kfold cv of 10 the mean result is negative (eg -0.001).

What most likely the cause of this negative result?

Thanks Jason.

Jason Brownlee February 1, 2019 at 5:36 am # REPLY

This is a common question that I answer here:
https://machinelearningmastery.com/faq/single-faq/why-are-some-scores-like-mse-negative-in-scikit-learn

Scott March 31, 2019 at 3:11 am # REPLY

Hello Jason:

I just want to say thank you for this blog. In the process of getting my Masters in machine learning I consult your articles with confidence that I will walk away with some value that will assist in my current and future classes. Keep doing what you do as there is much support from me!

Jason Brownlee March 31, 2019 at 9:31 am # REPLY

Thanks Scott, I'm very happy to hear that!

Li Tian April 2, 2019 at 11:38 am #

Thanks a lot for your summary, superb work. This is w
rate is certainly a key factor for gaining the better performance.
more complex your model is, the more carefully you should trea
performance , the most economic step is to change your learni

Definitely recommended!

Jason Brownlee April 2, 2019 at 2:20 pm # REPLY

Interesting finding, thanks for sharing!

Suraj Padhy May 8, 2019 at 5:44 pm # REPLY

"At extremes, a learning rate that is too large will result in weight updates that will be too large and the performance of the model (such as its loss on the training dataset) will oscillate over training epochs. Oscillating performance is said to be caused by weights that diverge (are divergent). A learning rate that is too small may never converge or may get stuck on a suboptimal solution."

In the above statement can you please elaborate on what it means when you say performance of the model will oscillate over training epochs?

Thanks in advance.

Suraj Padhy May 8, 2019 at 5:47 pm # REPLY

and why it wont have the oscillation of performance when the training rate is low.

Jason Brownlee May 9, 2019 at 6:37 am # REPLY

Small updates to weights will results in small changes in loss.

Jason Brownlee May 9, 2019 at 6:37 am # REPLY

The weights will go positive/negative in large swings.

Skill of the model (loss) will likely swing with the large weight updates.

Dan Potter February 5, 2020 at 1:03 am # REPLY

Optimizers that have a step-size parameter typically rely on a gradient to determine the direction the parameters (network weights) need to be moved to minimiz
a move is made. When the moves are too big (step-size is
overshooting the minimum. If you plot this loss function as t
choppy.

This page http://www.onmyphd.com/?p=gradient.descent h
(step-size) slider to the right. All the steps are in the right di
start to overshoot the minimum by more significant amount
each step.

Jason Brownlee February 5, 2020 at 8:16 am #

REPLY

Thanks for sharing.

Deepanshu SIngh June 26, 2019 at 4:43 pm #

REPLY

Hello Jason,
How can we set our learning rate to increase after each epoch in adam optimizer.

~Thanks

Jason Brownlee June 27, 2019 at 7:44 am #

REPLY

Perhaps you can use a custom callback?

james July 12, 2019 at 9:38 pm #

REPLY

Hi Jason, Any comments and criticism about this: https://medium.com/@jwang25610/self-adaptive-tuning-of-the-neural-network-learning-rate-361c92102e8b please?

Cheers

Jason Brownlee July 13, 2019 at 6:54 am #

REPLY

Perhaps you can summarize the thesis of the post for me?

PN July 22, 2019 at 8:58 am #

REPLY

Hi Jason,

Thank you very much for your posts, they are highly informative

In another post regarding tuning hyperparameters, somebody asked what order of hyperparameters is best to tune a network and your response was the learning rate.

I am training an MLP, and as such the parameters I believe I need to tune include the number of hidden layers, the number of neurons in the layers, activation function, batch size, optimizer because I feel I had read before that Adam is a decen

At the end of this article it states that if there is time, tune the le or the batch size/epoch/layer specific parameters first?

Jason Brownlee July 22, 2019 at 2:02 pm #

Yes, learning rate and model capacity (layers/node

Turyal August 20, 2019 at 8:52 pm #

REPLY

Hi Jason how to calculate the learning rate of scaled conjugate gradient algorithm ? What are sigma and lambda parameters in SCG algorithm ? Please reply

Jason Brownlee August 21, 2019 at 6:40 am #

REPLY

Not sure off the cuff, I don't have a tutorial on that topic. Perhaps start here:
https://en.wikipedia.org/wiki/Conjugate_gradient_method

Wills August 22, 2019 at 10:33 pm #

REPLY

Hi, it was a really nice read and explanation about learning rate. I have one question though. Should the learning rate be reset if we retrain a model. For example in a cnn, i use LR Decay that drop 0.5 every 5 epoch. (adam, initial lr = 0.001). I trained it for 50 epoch. If i want to add some new data and continue training, would it makes sense to start the LR from 0.001 again?

Jason Brownlee August 23, 2019 at 6:27 am #

REPLY

That is a tough question, nice one!

A lower learning rate should probably be used. Maybe as small as the final learning rate, but probably a little higher.

Maybe run some experiments to see what works best for your data and model?

NA October 22, 2019 at 2:26 pm #

REPLY

Thank you so much for your helpful posts,
I have one question about: How to use tf.contrib.keras.optimize

Jason Brownlee October 23, 2019 at 6:27 am #

REPLY

Sorry, I don't have tutorials on using tensorflow dir

What problem are you having exactly?

Lico October 29, 2019 at 2:28 am #

Thank you Jason for your sharing.

I am just wondering is it possible to set higher learning rate for
when training classification on an imbalanced dataset?

It looks like the learning rate is the same for all samples once it is set.

Thanks!

Jason Brownlee October 29, 2019 at 5:29 am # REPLY

Generally no. Good training requires that each batch has a mix of examples from each class.

Modifying the class weight is a good start.
Also oversampling the minority and undersampling the majority does well.

The best tip is to carefully choose the performance metric based on what type of predictions you need (crisp classes or probabilities, and if you have a cost matrix).

Jarek December 4, 2019 at 5:17 pm # REPLY

I have recently realized that we can choose learning rate to minimize parabola in one step: (theta,g) are in line for it, so we can e.g. use division of their standard deviations (more details: 5th page in https://arxiv.org/pdf/1907.07063):

learnig rate = sqrt(var(theta) / var(g))

what requires maintaining four (exponential moving) averages: of theta, theta², g, g². Is there considered 2nd order adaptation of learning rate in literature?

Jason Brownlee December 5, 2019 at 6:39 am # REPLY

Thanks for sharing.

GEBREYOHANNES ABRHA January 15, 2020 at 11:31 pm # REPLY

currently I am doing the LULC simulation using ANN based cellular Automata, but while I am trying to do ANN learning process am introuble how to decide the following
1. number of sample
2. neighborhood
3. learning rate
4. maximum iteration
5. momentum
so would you please help me how get ride of this challenge.
regards!

Jason Brownlee January 16, 2020 at 6:18 am #

Perhaps test a suite of different configurations to c

Deyvid Gueorguiev February 17, 2020 at 1:31 pm # REPLY

Hi, I found this page very helpful but I am still struggling with the following task.I have to improve an XOR's performance using NN and I have to use Matlab for that ,which I don't know much about. I have changed the gradient decent to an adaptive one with momentum called traingdx but im not sure how to change the values so I can get an optimal solution. This is the task https://hastebin.com/epatihayor.shell

Jason Brownlee February 18, 2020 at 6:13 am # REPLY

Perhaps the suggestions here will give you ideas:
http://machinelearningmastery.com/improve-deep-learning-performance/

Sathya June 23, 2020 at 11:53 am # REPLY

Hi Jason
I have a doubt .can we set learning rate schedule/decay mechanism in Adam optimizer…

Jason Brownlee June 23, 2020 at 1:28 pm # REPLY

No, adam is adapting the rate for you. Use SGD.

Sathya June 23, 2020 at 12:03 pm # REPLY

Hi Jason,
Can we change the architecture of lstm by adapting Ebbinghaus forgetting curve…

Jason Brownlee June 23, 2020 at 1:29 pm # REPLY

No idea sorry.

Sathya June 23, 2020 at 4:48 pm # REPLY

Thank you Jason..

Jason Brownlee June 24, 2020 at 6:23 am #

You're welcome.

Sathya June 23, 2020 at 4:50 pm #

REPLY

Thank you..

Jason Brownlee June 24, 2020 at 6:23 am #

REPLY

You're welcome.

RaviTeja August 8, 2020 at 12:29 am #

REPLY

Hi Jason your blog post are really great.

Lately I am trying to implement a research paper, for this paper the learning rate should reduce by a factor of 0.5 if validation perplexity hasn't improved after each epoch . For this i am trying to implement LearningRateScheduler (tensorflow, keras) callback but I am not able to figure this out. How to access validation loss inside the callback and also I am using custom training .
Due to the model architecture I cannot use tf.keras.Model.fit() method.

RaviTeja August 8, 2020 at 12:30 am #

REPLY

I am using SGD as optimizer.

Jason Brownlee August 8, 2020 at 6:02 am #

REPLY

Thanks!

Perhaps you can use the examples here as a starting point:
https://machinelearningmastery.com/using-learning-rate-schedules-deep-learning-models-python-keras/

Nikhil September 5, 2020 at 6:35 am #

Hi Jason,
Thank you for such an informative blog post on learning rate.
I didn't understand the term sub-optimal final set of weights in below line(Under Effect of learning rate) :
a large learning rate allows the model to learn faster, at the cos
Could you please explain what does it mean?
Thanks in advance.

Jason Brownlee September 5, 2020 at 6:55 am #

It means not the best model.

We give up some model skill for faster training.

Leave a Reply

Name (required)

Email (will not be published) (required)

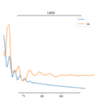
Website

SUBMIT COMMENT

Welcome!
I'm *Jason Brownlee* PhD
and I **help developers** get results with **machine learning**.
Read more

Never miss a tutorial:

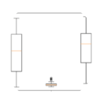
Picked for you:



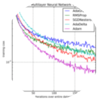
How to use Learning Curves to Diagnose Machine Learning Model Performance

Stacking Ensemble for Deep Learning Neural Networks in Py

How To Improve Deep Learning Performance



How to use Data Scaling Improve Deep Learning Model Stab



Gentle Introduction to the Adam Optimization Algorithm for Deep Learning